Birla Institute of Technology, Mesra

Off-Campus Jaipur

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Minor Project Report — B.Tech., Semester 7 (4th Year)

TripSolve

An Optimization-Grade Group Itinerary Planner for Jaipur Tourism

(Team Orienteering Problem with Time Windows, solved via Google OR-Tools, with LLM-assisted parsing and explainability)

Submitted by

Vidushi Himmatramka (Roll No. BTECH/25139/23)

Vedanshi Goyal (Roll No. BTECH/25167/23)

Class: CSE-7B

Under the Guidance of

Dr. Seema Gaur

Assistant Professor

CSE-BIT JAIPUR



Department of Computer Science & Engineering

Birla Institute of Technology, Mesra — Off-Campus Jaipur

MO-2026

Certificate

This is to certify that the Software Requirements Specification (SRS) Report entitled “TripSolve: An Optimization-Grade Group Itinerary Planner for Jaipur Tourism” has been prepared by Vidushi Himmatramka (BTECH/25139/23) and Vedanshi Goyal (BTECH/25167/23), students of Class CSE-7B, in partial fulfilment of the requirements for the Minor Project of Semester 7, B.Tech. in Computer Science & Engineering, at Birla Institute of Technology, Mesra, Off-Campus Jaipur.

The work presented in this report is a record of the candidates' own study carried out under my supervision and guidance, and has not been submitted elsewhere for the award of any other degree or diploma.

_____

Dr. Seema Gaur
Faculty Mentor
Department of Computer Science & Engineering, BIT Mesra, Off-Campus Jaipur

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of TripSolve, an optimization-grade group itinerary planning system for tourists visiting Jaipur. The purpose of this document is to provide a complete and unambiguous description of the system's intended behaviour so that it can serve as the basis for design, implementation, and testing during the development of the Minor Project (Semester 7). It is intended to be read by the project's faculty mentor, evaluators, and the development team itself.

1.2 Scope

TripSolve is a web-based application that allows a single traveler, or a group of travelers, to specify a time budget, a starting point, and a set of interests or must-visit attractions in and around Jaipur, and receive in return a single, time-window-feasible, near-optimal sightseeing route. Unlike existing itinerary generators that rely purely on a language model to produce a "plausible-looking" plan, TripSolve grounds its route generation in a mathematical optimization model — the Team Orienteering Problem with Time Windows (TOPTW) — solved using Google OR-Tools. A large language model (LLM) is used only for two well-scoped auxiliary tasks: (a) converting free-text user input into structured preference data, and (b) generating a plain-language explanation of the trade-offs behind the solver's chosen route. The system additionally supports multi-user group planning, in which the preferences of several travelers are combined into one shared, fair route.

The scope of this project, for the Minor Project deliverable, is limited to a working prototype seeded with real attraction data for Jaipur; the backend is designed to be city-agnostic so that additional cities can be added in future work.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
TOPTW	Team Orienteering Problem with Time Windows — a combinatorial optimization problem in which a fixed number of routes must visit a subset of locations, each with a time window and a 'score', so as to maximize total collected score without violating any time window.
OR-Tools	Google's open-source software suite for combinatorial optimization, used here to formulate and solve the TOPTW as a constraint program.
LLM	Large Language Model (e.g., Llama 3.3), used strictly for natural-language parsing and explanation generation, not for route decisions.
OSRM	Open Source Routing Machine — provides realistic road-network travel times between attractions.

PostGIS	A spatial database extension for PostgreSQL used to store and query attraction locations and geometries.
JWT	JSON Web Token — used for stateless user authentication.
POI	Point of Interest — a tourist attraction, monument, market, or eatery in the seeded dataset.
SRS	Software Requirements Specification — this document.
Time Window	The interval [open time, close time] during which a POI may be visited.
Utility Function	A mathematical function combining multiple travelers' preference scores for a POI into one shared score used by the solver.

1.4 References

1. Xie, J. et al., “TravelPlanner: A Benchmark for Real-World Planning with Language Agents,” ICML 2024.
2. Google OR-Tools Documentation — https://developers.google.com/optimization
3. Project Synopsis: “TripSolve: An Optimization-Grade Group Itinerary Planner for Jaipur Tourism,” BIT Mesra Off-Campus Jaipur, Minor Project Sem-7 Synopsis.
4. OSRM Project Documentation — https://project-osrm.org
5. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

1.5 Document Overview

The remainder of this document is organized as follows: Section 2 gives an overall description of the product, its functions, users, and constraints. Section 3 lists the specific functional and non-functional requirements in detail. Section 4 presents the system's structural and behavioural models (use cases, data flow, and architecture). Section 5 covers appendices, including the glossary and analysis models.

2. Overall Description

2.1 Product Perspective

TripSolve is a new, self-contained web application; it is not an add-on to any existing system. It is composed of four cooperating layers: a Next.js frontend for user interaction and map visualization, a FastAPI backend exposing REST endpoints, an optimization engine built on Google OR-Tools that solves the core TOPTW model, and a set of external/auxiliary services — OSRM for travel-time matrices, PostgreSQL/PostGIS for geospatial storage, Nominatim for geocoding, and a Groq-hosted or self-hosted LLM (e.g., Llama 3.3) for natural-language parsing and explanation.

2.2 Product Functions

At a high level, TripSolve performs the following functions:

- Accepts a traveler's (or group's) available time window, starting location, and interests, either through structured form input or free-text input parsed by the LLM layer.
- Maintains a seeded database of Jaipur attractions with location, opening/closing hours, average visit duration, and category tags.
- Computes realistic travel times between all relevant attraction pairs using OSRM road-network routing (not straight-line distance).
- For a group, combines each member's individual preference scores into a single shared multi-objective utility score per attraction.
- Formulates and solves a Team Orienteering Problem with Time Windows using OR-Tools to select and sequence the subset of attractions that maximizes collected utility while respecting all opening/closing windows and the total time budget.
- Returns a single, time-window-feasible, near-optimal route, rendered on an interactive Leaflet.js map with draggable stops.
- Uses the LLM layer to generate a plain-language explanation of key trade-offs the solver made (e.g., why one attraction was prioritized over another, or why a requested stop could not be included).
- Allows authenticated users to save, retrieve, and share planned trips.

2.3 User Classes and Characteristics

User Class	Characteristics
Solo Traveler	A single tourist who inputs their available time and interests and expects one optimized route. No technical background assumed.
Group Organizer	Creates a shared trip, invites other group members, and reviews/finalizes the combined route. Needs visibility into how conflicting preferences were resolved.
Group Member	Submits personal interests/preferences into a shared trip created by the organizer; may not interact with the optimization logic directly.
Guest User	Can generate a one-off itinerary without creating an account, but cannot save or share trips (no JWT-authenticated session).
Project Evaluator / Mentor	Reviews system outputs, the evaluation report comparing TripSolve against a pure-LLM baseline, and the explanation quality.

2.4 Operating Environment

- Client: Any modern desktop or mobile web browser (Chrome, Firefox, Edge, Safari) supporting HTML5 and JavaScript ES2020+.
- Frontend hosting: Vercel.
- Backend hosting: Render or Fly.io, running a containerized FastAPI + OR-Tools service.
- Database: PostgreSQL 14+ with the PostGIS extension, hosted alongside the backend or on a managed database service.
- Routing engine: A self-hosted or managed OSRM instance pre-loaded with the Jaipur road network extract from OpenStreetMap.
- LLM inference: Groq-hosted Llama 3.3 API, or a self-hosted equivalent, reachable over HTTPS from the backend.

2.5 Design and Implementation Constraints

- The final routing/sequencing decision must always be produced by the OR-Tools solver; the LLM must never be permitted to alter, override, or directly generate the itinerary sequence.
- The system must remain city-agnostic in its backend data model, even though this Minor Project seeds only Jaipur attraction data.
- Travel-time computation must use real road-network data (via OSRM), not Euclidean/straight-line approximations.
- Authentication must be stateless (JWT-based) to keep the backend horizontally scalable.
- The solver must return a result within an interactive time budget (see Non-Functional Requirements, Section 3.2) suitable for a web request-response cycle.

2.6 Assumptions and Dependencies

- Opening hours, visit durations, and locations for the seeded Jaipur attractions are assumed to be accurate as of the data collection date and are not assumed to change during the semester.
- The OSRM instance is assumed to have a complete and up-to-date Jaipur road-network extract.
- The chosen LLM (Llama 3.3, via Groq or self-hosted) is assumed to be available with acceptable latency for parsing and explanation generation.
- Users are assumed to have a basic internet connection sufficient to load an interactive map-based web application.
- The project assumes access to a small evaluation set of test scenarios for comparing TripSolve against a baseline pure-LLM itinerary generator.

3. Specific Requirements

3.1 Functional Requirements

The functional requirements below describe the observable behaviour the system shall exhibit. Each requirement is uniquely identified (FR-n) for traceability into design and test documents.

ID	Title	Description
FR-1	Interest & Constraint Input	The system shall allow a user to specify available start time, end time, starting location, and a set of interests/must-visit attractions, either via a structured form or free-text input.
FR-2	NL Parsing (LLM)	The system shall use the LLM layer to parse free-text preference input into structured JSON (interest tags, hard constraints, priority weights) consumable by the optimization engine.
FR-3	Attraction Database	The system shall maintain a database of Jaipur POIs, each with geolocation, opening/closing time windows, average visit duration, and category tags, stored in PostgreSQL/PostGIS.
FR-4	Travel-Time Matrix	The system shall query OSRM to compute a real road-network travel-time matrix between all candidate POIs and the user's starting location.
FR-5	Group Preference Aggregation	For group trips, the system shall combine each member's individual preference scores for each POI into one shared multi-objective utility score, using a documented, fair aggregation function.
FR-6	TOPTW Route Optimization	The system shall formulate the attraction-selection and sequencing problem as a Team Orienteering Problem with Time Windows and solve it using Google OR-Tools, subject to the user's/group's time budget and each POI's opening/closing window.
FR-7	Feasibility Guarantee	The system shall return only routes in which every visited POI is entered within its open/close time window; infeasible POIs shall be excluded rather than scheduled in violation.
FR-8	Route Visualization	The system shall render the optimized route on an interactive Leaflet.js map, showing ordered stops, connecting paths, and arrival/departure times, with stops adjustable by drag-and-drop.
FR-9	Re-optimization on Edit	If a user manually drags/reorders a stop or removes a POI, the system shall re-run the solver (or a fast local repair) and report whether the edited route remains time-window-feasible.
FR-10	Explanation Generation (LLM)	The system shall use the LLM layer to generate a plain-language explanation of key trade-offs behind the solver's chosen route (e.g., why a POI was dropped or reordered).
FR-11	User Authentication	The system shall allow users to register and log in using JWT-based authentication to save and later retrieve their trips.
FR-12	Trip Persistence & Sharing	Authenticated users shall be able to save a generated trip and share a read-only or editable link with other group members.
FR-13	Guest Mode	The system shall allow unauthenticated (guest) users to generate a one-off itinerary without the ability to save or share it.
FR-14	Baseline Comparison (Evaluation)	The system shall support running the same test scenario through both the OR-Tools-based planner and a baseline pure-LLM itinerary

		generator, and log metrics (total travel time, time-window violations) for comparison.

3.2 Non-Functional Requirements

3.2.1 Performance

- The OR-Tools solver shall return a feasible or near-optimal route for a typical query (up to ~30 candidate POIs, one day time budget) within 5 seconds under normal server load.
- The travel-time matrix for a given query shall be retrievable from OSRM within 2 seconds, using pre-computed/cached matrices where the candidate POI set is unchanged.
- The frontend map view shall render an optimized route of up to 15 stops within 2 seconds of receiving the backend response.

3.2.2 Reliability & Correctness

- The system shall never output a route that schedules arrival at a POI outside its declared opening/closing window (hard constraint, non-negotiable).
- If the LLM parsing layer fails or returns malformed output, the system shall fall back to the structured form input and shall not silently pass invalid data to the solver.

3.2.3 Usability

- A first-time user shall be able to generate a valid single-day itinerary within 3 user interactions (enter time budget → enter interests → view route) without consulting external help.
- Explanations generated by the LLM layer shall be written in plain, non-technical language understandable to a non-expert tourist.

3.2.4 Scalability & Maintainability

- The backend data model and API shall remain city-agnostic, so that a new city can be supported by adding a new POI dataset without changing the optimization or explanation logic.
- The optimization engine, LLM layer, and routing layer shall be decoupled behind clear internal API boundaries so that any one component (e.g., the LLM provider) can be swapped independently.

3.2.5 Security

- All authenticated endpoints shall require a valid JWT; expired or tampered tokens shall be rejected with an appropriate HTTP 401 response.
- User credentials shall be stored using industry-standard password hashing (e.g., bcrypt/argon2); passwords shall never be logged or stored in plain text.

3.2.6 Availability

- The deployed prototype shall target at least 95% uptime during the project evaluation period, consistent with a free/low-cost hosting tier on Vercel and Render/Fly.io.

3.3 External Interface Requirements

3.3.1 User Interfaces

- A landing/input page for entering trip parameters (dates, time budget, starting point, interests) via form fields or a free-text box.
- A group-trip page allowing an organizer to invite members and each member to submit their own preferences.
- An interactive map/itinerary page (Leaflet.js) showing the ordered route with draggable stops, timing annotations, and an accompanying plain-language explanation panel.

- An authentication (login/register) page and a “my trips” dashboard for saved itineraries.

3.3.2 Hardware Interfaces

The system has no direct hardware interfaces; it relies on standard client devices (desktop/laptop/mobile) with a browser and internet connectivity, and standard cloud server infrastructure for the backend.

3.3.3 Software Interfaces

Component	Interface / Protocol
Frontend ↔ Backend	REST/JSON over HTTPS (FastAPI endpoints).
Backend ↔ OSRM	HTTP requests to the OSRM ‘table’ and ‘route’ services for travel-time/distance matrices and path geometry.
Backend ↔ PostgreSQL/PostGIS	SQL over a database driver (e.g., SQLAlchemy/psycopg), including spatial queries.
Backend ↔ LLM Service	HTTPS API calls to the Groq-hosted (or self-hosted) Llama 3.3 endpoint for parsing and explanation generation.
Backend ↔ Nominatim	HTTP requests for geocoding free-text place names during data seeding/administration.
Backend ↔ OR-Tools	In-process library calls (Python) to the OR-Tools constraint-programming solver; no network hop.

3.3.4 Communication Interfaces

All client–server and inter-service communication shall occur over HTTPS/TLS. JWT tokens shall be transmitted using the Authorization: Bearer header. The system does not require any real-time (WebSocket) channel for the Minor Project scope, though it is a candidate future enhancement for live re-optimization.

4. System Models

4.1 Use Case Model

The table below summarizes the primary use cases supported by TripSolve; a corresponding use-case diagram (Traveler / Group Organizer / Group Member / Evaluator as actors, connected to the use cases below) will be included in the System Design Specification.

ID	Use Case	Actor(s)	Description
UC-1	Plan Solo Trip	Traveler	User enters time budget, start location, and interests; system returns an optimized single-traveler route with explanation.
UC-2	Plan Group Trip	Group Organizer, Group Members	Organizer creates a shared trip and invites members; each member submits preferences; system aggregates preferences and returns one shared route.
UC-3	Edit Route Manually	Traveler	User drags a stop or removes a POI on the map; system re-checks/re-optimizes and reports feasibility.
UC-4	View Explanation	Traveler	User requests or is shown the LLM-generated plain-language rationale for the chosen route and any excluded POIs.
UC-5	Save / Share Trip	Authenticated User	User saves a generated itinerary to their account and shares a link with co-travelers.
UC-6	Register / Login	Any User	User creates an account or logs in to obtain a JWT for authenticated features.
UC-7	Run Baseline Comparison	Project Evaluator	Evaluator runs a fixed test scenario through both the OR-Tools planner and the pure-LLM baseline and reviews logged metrics.

4.2 High-Level Data Flow

At a high level, data flows through the system as follows:

6. The user (or group members) submit trip parameters and interests through the Next.js frontend.
7. Free-text input, if used, is sent to the LLM parsing service and converted into structured preference data.
8. The backend retrieves candidate POIs from PostgreSQL/PostGIS based on the user's interests, location radius, and opening hours overlapping the requested time budget.
9. The backend requests a travel-time matrix between the starting location and all candidate POIs from OSRM.
10. For group trips, individual preference scores are combined into one shared utility score per POI.
11. The TOPTW model (POIs, time windows, travel times, utility scores, time budget) is passed to the OR-Tools solver, which returns the selected, ordered subset of POIs.
12. The solved route is sent to the LLM explanation service, which generates a plain-language summary of key trade-offs.
13. The route, timing, and explanation are returned to the frontend and rendered on the Leaflet.js map.

14. If the user is authenticated, the finalized trip may be persisted to the database for later retrieval or sharing.

4.3 System Architecture Overview

TripSolve follows a layered client–server architecture:

- Presentation Layer: Next.js + Leaflet.js — trip input forms, interactive map, explanation panel, authentication views.
- API Layer: FastAPI — exposes REST endpoints for trip creation, group preference submission, route retrieval, editing, and authentication.
- Optimization Layer: Python service wrapping Google OR-Tools — builds and solves the TOPTW constraint program for each request.
- AI Layer: LLM client module — invoked strictly for (a) parsing free-text input and (b) generating natural-language explanations; never invoked for route decisions.
- Data Layer: PostgreSQL + PostGIS — stores POIs, users, trips, and group preferences; supports spatial queries for candidate POI selection.
- External Services: OSRM (routing/travel time), Nominatim (geocoding, used during data seeding).

A detailed component diagram, sequence diagrams for the solo- and group-trip flows, and the entity–relationship diagram for the data layer will be elaborated in the accompanying System Design Specification (SDS) document, as this SRS is scoped to requirements rather than detailed design.

4.4 Evaluation Plan

To validate the claimed benefit of the optimization-grounded approach, the project will run an identical set of test scenarios (varying number of POIs, group size, and time budgets) through (a) the TOPTW/OR-Tools pipeline described in this document and (b) a baseline pure-LLM itinerary generator, and report, for each: total travel time, number of time-window violations, and number of requested POIs successfully included, in line with the expected outcomes stated in the project synopsis.

5. Appendices

5.1 Analysis Models

Use-case diagrams, sequence diagrams, the entity–relationship diagram for the PostGIS schema, and the OR-Tools constraint-program formulation (decision variables, objective function, and constraints for the TOPTW) will be documented in detail in the System Design Specification (SDS) that follows this SRS.

5.2 To-Be-Determined (TBD) List

- Exact list and count of seeded Jaipur POIs to be finalized after the data-collection phase.
- Precise weighting scheme for aggregating group members' preference scores (e.g., simple average vs. fairness-constrained aggregation) to be finalized during design.
- Choice between Groq-hosted and self-hosted Llama 3.3 for production deployment, pending cost/latency evaluation.

5.3 Team & Supervision

Role	Name	Details
Faculty Mentor	Dr. Seema Gaur	Department of CSE, BIT Mesra, Off-Campus Jaipur
Team Member 1	Vidushi Himmatramka	Roll No. BTECH/25139/23, Class CSE-7B
Team Member 2	Vedanshi Goyal	Roll No. BTECH/25167/23, Class CSE-7B